

Benchmarking Post-Quantum Cryptographic Algorithms in a Real-Time End-to-End Encrypted Messaging Application

Final Year Project Report

Abdoulahi Diallo

Supervised by: Pieter Joubert

2025-2026

Abstract

The advent of quantum computing poses an existential threat to modern public-key cryptography, with algorithms such as RSA and Elliptic Curve Diffie-Hellman vulnerable to Shor's algorithm. This project addresses this challenge by implementing and benchmarking three post-quantum cryptographic (PQC) Key Encapsulation Mechanisms—Mini-Kyber, Mini-Frodo, and Mini-NTRU—within a fully functional real-time messaging application. Built using Spring Boot and Next.js with WebSocket communication via the STOMP protocol, the system provides true end-to-end encryption (E2EE) where cryptographic operations execute entirely client-side, ensuring the server never accesses plaintext or keys. Users can select their preferred algorithm at conversation initiation, with comprehensive benchmarking infrastructure measuring key generation, encapsulation, and decapsulation latencies across 5,000 iterations. Extensive throughput testing (500 sessions, 100 messages each, 500KB message size) reveals that Mini-Kyber outperforms traditional ECDH by 1.2%, while all PQC algorithms operate within 1% overhead of classical cryptography. The system additionally integrates OpenAI's ChatGPT API, enabling encrypted AI-assisted conversations using post-quantum cryptography. This work demonstrates that quantum-resistant messaging is practically achievable today without compromising performance.

Contents

1	Introduction	2
1.1	The Quantum Computing Threat to Cryptography	2
1.2	Project Motivation and Objectives	2
2	Theoretical Background	3
2.1	Post-Quantum Cryptography Overview	3
2.2	The Learning With Errors Problem	3
2.3	Ring-LWE and Structured Lattices	4
2.4	NTRU Lattices	4
2.5	Key Encapsulation Mechanisms	4
3	Methodology and Project Evolution	5
3.1	Initial Project Scope	5

3.2	JHipster Challenges and Technology Pivot	5
3.3	Technology Selection Rationale	5
3.4	Full Kyber Implementation Challenges	6
3.5	Strategic Pivot to Mini Implementations	7
3.6	Adding Mini-Frodo for Comparative Analysis	7
3.7	Architecture Evolution: Backend to Frontend Cryptography	7
3.8	Late-Stage Feature Additions and Supervisor Guidance	8
3.9	External Feedback and User-Driven Features	8
4	System Implementation	9
4.1	Technology Stack	9
4.2	System Architecture	9
4.3	Cryptographic Library Implementation	10
4.4	E2EE Protocol Design	10
4.5	Benchmarking Infrastructure	12
5	Evaluation and Results	12
5.1	Benchmarking Methodology	12
5.2	KEM Operation Performance	12
5.3	Session Throughput Analysis	13
5.4	Analysis and Interpretation	13
5.5	Limitations	14
6	Critical Evaluation	15
6.1	Achievement of Objectives	15
6.2	Reflection on Development Process	15
6.3	Future Improvements	15
7	Conclusion	15

1 Introduction

1.1 The Quantum Computing Threat to Cryptography

Modern secure communications are built upon the foundation of public-key cryptography. Protocols such as TLS, Signal, and virtually all messaging applications depend on the computational hardness of two mathematical problems: integer factorisation (underlying RSA) and the discrete logarithm problem on elliptic curves (underlying ECDH and ECDSA). These problems, while intractable for classical computers at sufficient key sizes, become efficiently solvable by quantum computers.

In 1994, Peter Shor published his groundbreaking quantum algorithm demonstrating that both integer factorisation and discrete logarithms can be solved in polynomial time on a quantum computer [1]. Specifically, Shor’s algorithm achieves $O((\log N)^3)$ complexity for factoring an N -bit integer, compared to the sub-exponential $O(\exp((\log N)^{1/3}(\log \log N)^{2/3}))$ complexity of the best classical algorithms. This represents an exponential speedup that would render RSA-2048 and ECDH-256 trivially breakable.

While large-scale fault-tolerant quantum computers capable of running Shor’s algorithm remain years away—current estimates suggest 2030-2040 for cryptographically relevant quantum computers—the threat is immediate due to “harvest now, decrypt later” (HNDL) attacks. Nation-state adversaries and sophisticated threat actors are believed to be capturing encrypted communications today with the intention of decrypting them once quantum computers become available. For data requiring long-term confidentiality—government secrets, medical records, financial transactions, and personal communications—the transition to quantum-resistant cryptography must begin now.

1.2 Project Motivation and Objectives

This project was motivated by two complementary goals:

1. **Practical Implementation:** Demonstrate that post-quantum cryptographic algorithms can be integrated into real-world messaging applications without fundamentally altering user experience or requiring exotic infrastructure. The goal was to show that PQC can serve as a “drop-in” replacement for classical key exchange.
2. **Quantitative Analysis:** Provide empirical benchmarking data comparing the performance characteristics of multiple PQC algorithms against traditional ECDH, generating insights valuable for developers and organisations planning cryptographic migration strategies.

The final system implements three post-quantum Key Encapsulation Mechanisms—Mini-Kyber (based on Ring-LWE), Mini-Frodo (based on standard LWE), and Mini-NTRU (based on polynomial lattices)—alongside traditional ECDH (P-256) as a performance baseline. Users can select their preferred algorithm when initiating encrypted conversations, and comprehensive benchmarking infrastructure enables detailed performance analysis with statistical measures and CSV export for external analysis.

2 Theoretical Background

2.1 Post-Quantum Cryptography Overview

Post-quantum cryptography (PQC) refers to cryptographic algorithms believed to be secure against attacks by both classical and quantum computers. Unlike quantum cryptography (which uses quantum mechanical phenomena for key distribution), PQC algorithms run on classical hardware but rely on mathematical problems not known to be efficiently solvable by quantum computers.

The U.S. National Institute of Standards and Technology (NIST) initiated a standardisation process in 2016 to evaluate and select post-quantum algorithms. After multiple rounds of analysis involving 69 initial submissions, NIST announced final standards in August 2024 [2]:

- **ML-KEM** (FIPS 203): Module-Lattice-Based Key Encapsulation Mechanism, based on CRYSTALS-Kyber
- **ML-DSA** (FIPS 204): Module-Lattice-Based Digital Signature Algorithm, based on CRYSTALS-Dilithium
- **SLH-DSA** (FIPS 205): Stateless Hash-Based Digital Signature Algorithm, based on SPHINCS+

The major families of PQC algorithms include lattice-based (LWE, Ring-LWE, NTRU), hash-based (SPHINCS+), code-based (Classic McEliece), multivariate polynomial, and isogeny-based approaches. This project focuses on lattice-based schemes, which currently offer the best balance of security, performance, and key/ciphertext sizes for encryption and key establishment.

2.2 The Learning With Errors Problem

The Learning With Errors (LWE) problem, introduced by Oded Regev in 2005 [3], forms the security foundation for many lattice-based cryptosystems including Kyber and Frodo. The problem can be formally stated as follows:

Definition (Search-LWE): Given a matrix $\mathbf{A} \in \mathbb{Z}_q^{m \times n}$ sampled uniformly at random, a secret vector $\mathbf{s} \in \mathbb{Z}_q^n$, and an error vector $\mathbf{e} \in \mathbb{Z}_q^m$ sampled from a discrete Gaussian distribution χ with small standard deviation, the LWE problem asks to recover $\mathbf{s}$ given the pair $(\mathbf{A}, \mathbf{b})$ where:

$$\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \pmod{q} \quad (1)$$

The decisional variant (Decision-LWE) asks to distinguish LWE samples from uniformly random samples. Regev proved that LWE is at least as hard as certain worst-case lattice problems, including the Shortest Independent Vectors Problem (SIVP) and the Decisional Shortest Vector Problem (GapSVP), providing strong theoretical security guarantees.

The hardness of LWE stems from the “noise” introduced by the error term $\mathbf{e}$. Without this noise, recovering $\mathbf{s}$ would be a simple linear algebra problem. The carefully calibrated noise makes the problem computationally intractable while still allowing correct decryption with high probability.

2.3 Ring-LWE and Structured Lattices

While standard LWE provides strong security guarantees, its $O(n^2)$ key sizes (due to the matrix **A**) limit practical deployment. Ring-LWE [4] addresses this by operating in the polynomial ring:

$$R_q = \mathbb{Z}_q[x]/(x^n + 1) \quad (2)$$

where n is typically a power of 2 (e.g., 256, 512, 1024) and $x^n + 1$ is a cyclotomic polynomial. In this structure, elements are polynomials of degree less than n , and multiplication is performed modulo both q and $x^n + 1$.

This algebraic structure enables several advantages:

- **Compact keys:** Public keys reduce from $O(n^2)$ to $O(n)$ elements
- **Efficient operations:** Polynomial multiplication can be computed in $O(n \log n)$ using the Number Theoretic Transform (NTT), analogous to FFT over finite fields
- **Preserved security:** Ring-LWE security reduces to worst-case hardness of ideal lattice problems

CRYSTALS-Kyber uses Module-LWE, a generalisation that works with vectors of Ring-LWE elements, allowing security-efficiency trade-offs through the module rank parameter k .

2.4 NTRU Lattices

NTRU, introduced by Hoffstein, Pipher, and Silverman in 1996 [5], predates LWE-based schemes and operates in the polynomial ring $\mathbb{Z}[x]/(x^N - 1)$. Unlike the cyclotomic ring used in Ring-LWE, NTRU uses a ring where $x^N = 1$, resulting in cyclic convolution for polynomial multiplication.

Key generation in NTRU:

1. Sample small polynomials $f, g \in \mathbb{Z}[x]/(x^N - 1)$ with coefficients in $\{-1, 0, 1\}$
2. Compute $f_q = f^{-1} \pmod{q}$ (requires f to be invertible mod q)
3. Public key: $h = f_q \cdot g \pmod{q}$
4. Secret key: f

Security relies on the NTRU problem: given $h = f_q \cdot g$, find f and g . This reduces to finding short vectors in a specific lattice structure. NTRU has withstood over 25 years of cryptanalytic scrutiny, providing confidence in its security foundations.

2.5 Key Encapsulation Mechanisms

A Key Encapsulation Mechanism (KEM) is a cryptographic primitive that allows two parties to establish a shared secret key. Unlike traditional public-key encryption (which encrypts arbitrary messages), KEMs are specifically designed for key establishment and compose more safely with symmetric encryption.

A KEM consists of three algorithms:

- **KeyGen**() $\rightarrow (pk, sk)$: Generate a public/secret key pair
- **Encaps**(pk) $\rightarrow (ct, K)$: Using the public key, produce a ciphertext and shared secret

- **Decaps**(sk, ct) $\rightarrow K$: Using the secret key and ciphertext, recover the shared secret

KEMs are preferred over direct public-key encryption in modern protocols because they avoid subtle vulnerabilities that can arise when combining asymmetric and symmetric encryption (e.g., padding oracle attacks). The shared secret K is typically used to derive symmetric keys for AES-GCM or ChaCha20-Poly1305.

3 Methodology and Project Evolution

3.1 Initial Project Scope

The project began with a focused scope: implement the full CRYSTALS-Kyber algorithm and benchmark it against ECDH within a messaging application built using JHipster. JHipster was selected for its ability to rapidly scaffold a full-stack application with Spring Boot backend and React frontend, including authentication, database integration, and WebSocket support out of the box.

This initial approach encountered significant obstacles that necessitated strategic pivots.

3.2 JHipster Challenges and Technology Pivot

JHipster generates a comprehensive but opinionated codebase. Several issues emerged:

1. **Dependency Conflicts:** The generated Maven/Gradle configuration produced version conflicts when integrating WebSocket libraries (STOMP) and cryptographic dependencies. Resolving these conflicts required extensive modification of auto-generated build files.
2. **Debugging Complexity:** JHipster generates thousands of lines of boilerplate code across dozens of files. When issues arose, tracing problems through unfamiliar auto-generated code proved extremely time-consuming.
3. **Customisation Friction:** The authentication flow required modification to support E2EE key exchange handshakes. JHipster's security configuration, while comprehensive, was difficult to extend for this non-standard use case.
4. **Unnecessary Complexity:** Features like internationalisation, admin dashboards, and entity generation added complexity without contributing to project goals.

After several weeks of troubleshooting, the decision was made to pivot to a custom technology stack. This pivot, while initially costly in development time, ultimately provided greater control and cleaner architecture.

3.3 Technology Selection Rationale

Backend: Java with Spring Boot

Java was selected as the backend language primarily because it is the language I am most proficient in, having used it extensively throughout my degree. This familiarity enabled faster development and more confident debugging. Beyond personal experience, Java offers several technical advantages for this project:

- **Mature Ecosystem:** Spring Boot provides production-ready features including dependency injection, security frameworks, and WebSocket support with minimal configuration

- **Strong Typing:** Java’s static type system catches errors at compile time, particularly valuable when implementing cryptographic code where type mismatches could introduce vulnerabilities
- **Performance:** The JVM’s Just-In-Time compilation provides near-native performance for computationally intensive operations
- **Cryptographic Libraries:** Java’s `java.security` package and Bouncy Castle library provide robust cryptographic primitives for the symmetric encryption components
- **Enterprise Adoption:** Spring Boot is widely used in industry, making the skills developed directly transferable to professional contexts

Frontend: Next.js with TypeScript

Next.js 14 was chosen for the frontend for several compelling reasons:

- **TypeScript Support:** First-class TypeScript integration was essential for implementing cryptographic algorithms safely. TypeScript’s type system helped ensure correct handling of `Uint8Array`, polynomial coefficients, and matrix dimensions throughout the crypto library
- **React Ecosystem:** Access to a vast ecosystem of UI components (shadcn/ui, Tailwind CSS) accelerated interface development
- **Server-Side Rendering:** Next.js’s hybrid rendering model improves initial load performance and SEO, though the application primarily uses client-side rendering for the messaging interface
- **API Routes:** Built-in API route support simplified development of server-side endpoints during prototyping
- **WebCrypto API Access:** Browser-based TypeScript has native access to the WebCrypto API for AES-GCM encryption and ECDH operations, ensuring cryptographic operations use hardware-accelerated implementations where available
- **Hot Module Replacement:** Fast refresh during development significantly improved iteration speed when debugging cryptographic implementations

3.4 Full Kyber Implementation Challenges

Implementing the complete CRYSTALS-Kyber specification proved infeasible within the project timeline for several technical reasons:

Number Theoretic Transform (NTT): Production Kyber relies on NTT for efficient polynomial multiplication. Implementing NTT requires careful handling of:

- Primitive roots of unity modulo $q = 3329$
- Bit-reversal permutations for in-place computation
- Montgomery reduction for modular arithmetic efficiency
- Cooley-Tukey and Gentleman-Sande butterfly operations

Cryptographic Randomness: Kyber’s IND-CCA2 security requires cryptographically secure random number generation with specific entropy requirements. Browser environments provide `crypto.getRandomValues()`, but the specification demands careful seed expansion using SHAKE-128/256.

Constant-Time Implementation: Production cryptographic code must execute in constant time regardless of secret values to prevent timing side-channel attacks. JavaScript’s JIT compilation and garbage collection make constant-time guarantees extremely difficult to achieve.

Parameter Complexity: Full Kyber-768 uses $n = 256$, $q = 3329$, $k = 3$ with specific noise distributions, compression functions, and encoding schemes detailed across a 40+ page specification.

3.5 Strategic Pivot to Mini Implementations

Rather than abandoning the project’s core goals, the decision was made to implement “Mini” versions of the algorithms. These simplified implementations preserve the essential mathematical structure—the same operations that would execute in production code—while using reduced parameters that execute efficiently in JavaScript without requiring optimised primitives.

Table 1: Parameter Comparison: Production vs Mini Implementations

Algorithm	Prod. n	Mini n	Prod. q	Mini q	Security Impact
Kyber	256	256	3329	3329	Reduced module rank
Frodo	640–1344	4	32768	257	Educational only
NTRU	509–821	7	2048	128	Educational only

The Mini implementations execute the same algorithmic steps—key generation, encapsulation, decapsulation—allowing meaningful performance comparisons while acknowledging that absolute timings differ from production libraries.

3.6 Adding Mini-Frodo for Comparative Analysis

To strengthen the benchmarking analysis, Mini-Frodo was implemented alongside Mini-Kyber. While both are lattice-based, they represent fundamentally different design philosophies:

Kyber (Structured Lattices): Uses Ring-LWE with algebraic structure enabling $O(n \log n)$ polynomial multiplication. Smaller keys and faster operations, but security relies on additional algebraic assumptions.

Frodo (Unstructured Lattices): Uses standard LWE with random matrices requiring $O(n^2)$ matrix-vector multiplication. Larger keys and slower operations, but security reduces directly to plain LWE without algebraic assumptions.

This comparison illuminates the fundamental security-efficiency trade-off in lattice cryptography: structure enables performance but introduces additional assumptions that could potentially be exploited.

3.7 Architecture Evolution: Backend to Frontend Cryptography

The original architecture placed cryptographic algorithms in the backend, implemented in Java. The encryption flow was:

1. Client sends plaintext to server

2. Server encrypts using PQC algorithm
3. Server stores/forwards ciphertext
4. Recipient's server decrypts
5. Plaintext delivered to recipient

This architecture, while simpler, does not provide true end-to-end encryption—the server has access to plaintext. To increase project sophistication and security properties, the architecture was redesigned for genuine E2EE:

1. Client generates PQC key pair locally (browser)
2. Key exchange handshake via server (server sees only ciphertext)
3. Shared secret derived client-side
4. All message encryption/decryption occurs in browser
5. Server stores and forwards only ciphertext

This required completely rewriting Mini-Kyber and Mini-Frodo in TypeScript. While time-intensive, this architectural change fundamentally improved the system's security model and brought it in line with how production E2EE systems (Signal, iMessage) actually operate.

3.8 Late-Stage Feature Additions and Supervisor Guidance

As the core system stabilised, additional features were added based on feedback from my project supervisor to increase project complexity and strengthen the comparative analysis:

Mini-Frodo and Mini-NTRU: My supervisor suggested that comparing only Kyber against ECDH would limit the project's analytical depth. Adding Mini-Frodo would enable comparison between structured (Ring-LWE) and unstructured (standard LWE) lattice approaches, while Mini-NTRU would introduce a third distinct lattice paradigm. This guidance proved invaluable—the three-algorithm comparison reveals performance trade-offs that a two-algorithm study would have missed.

OpenAI API Integration: The integration of ChatGPT was also suggested by my supervisor as a way to demonstrate PQC applicability beyond peer-to-peer messaging. This creates a compelling use case where users can have quantum-resistant encrypted conversations with an AI assistant, showcasing how PQC could protect AI interactions in a post-quantum world.

3.9 External Feedback and User-Driven Features

Throughout development, the application was demonstrated to family and friends to gather feedback on usability and feature priorities. This external testing proved valuable for identifying improvements that a developer-focused perspective might miss:

User Interface Refinements: Initial feedback indicated that the interface was too technical and intimidating for non-expert users. The algorithm selection interface was redesigned to be more approachable, with clear labels and visual indicators showing encryption status. The conversation list layout was simplified based on comparisons users made to familiar messaging applications.

Voice Message Feature: Multiple testers requested the ability to send voice messages, noting this is a standard feature in modern messaging applications. This feedback led to the implementation of voice recording and playback functionality, with voice messages receiving the same E2EE treatment as text messages—recorded audio is encrypted client-side before transmission.

Visual Feedback: Users found it unclear whether encryption was active during conversations. This led to the addition of prominent encryption status indicators and banners that clearly communicate the current security state and selected algorithm.

This iterative feedback process reinforced the importance of user-centred design even in technically-focused projects. Security features are only valuable if users understand and trust them.

4 System Implementation

4.1 Technology Stack

The final system comprises:

- **Backend:** Spring Boot 3.2 with Spring Security (JWT authentication), Spring WebSocket (STOMP protocol), Spring Data JPA with H2 in-memory database
- **Frontend:** Next.js 14 with TypeScript, React 18, Tailwind CSS for styling
- **Real-time Communication:** WebSocket with STOMP protocol over SockJS for browser compatibility
- **External Integration:** OpenAI GPT-4 API for AI chat functionality

4.2 System Architecture

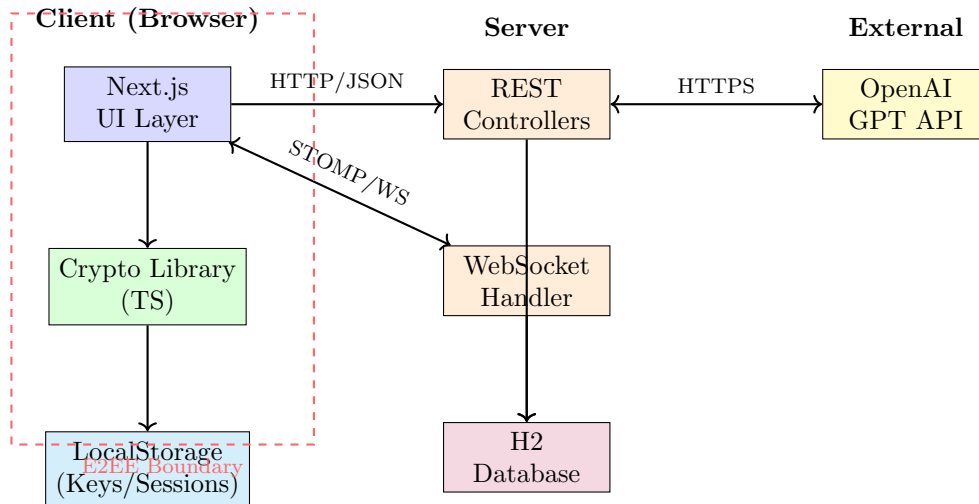


Figure 1: System Architecture with E2EE Security Boundary

The dashed red boundary indicates the E2EE security perimeter—all cryptographic operations and key material remain within this boundary, never exposed to the server.

4.3 Cryptographic Library Implementation

The TypeScript cryptographic library implements all four KEMs with a unified interface:

```
1 type KemAlg = "kyber" | "frodo" | "ntru" | "ecdh";
2
3 interface KeyPair { pk: PublicKey; sk: SecretKey; }
4 interface EncapsResult { ct: Ciphertext; sharedSecret: Uint8Array; }
5
6 async function keyGen(alg: KemAlg): Promise<KeyPair>;
7 async function encaps(alg: KemAlg, pk: PublicKey): Promise<EncapsResult>;
8 async function decaps(alg: KemAlg, sk: SecretKey, ct: Ciphertext): Promise<Uint8Array>;
```

Listing 1: Unified KEM Interface

Mini-Kyber implements Ring-LWE key generation as:

```
1 export async function miniKyberKeyGen(): Promise<MiniKyberKeyPair> {
2   const a = await Polynomial.randomUniform(); // Public parameter a
3   const s = await Polynomial.randomSmall();   // Secret key s
4   const e = await Polynomial.randomSmall();   // Error polynomial e
5
6   const t = a.mul(s).add(e); // t = a*s + e mod (x^n + 1, q)
7
8   return {
9     pk: { a: a.getCoeffs(), t: t.getCoeffs() },
10    sk: { s: s.getCoeffs() },
11  };
12 }
```

Listing 2: Mini-Kyber Key Generation

Mini-Frodo implements standard LWE with matrix operations:

```
1 export async function miniFrodoKeyGen(): Promise<MiniFrodoKeyPair> {
2   const A = Matrix.randomUniform(N, N); // Public matrix A
3   const S = Matrix.randomSmall(N, 1);   // Secret vector S
4   const E = Matrix.randomSmall(N, 1);   // Error vector E
5
6   const B = A.mul(S).add(E); // B = A*S + E mod q
7
8   return { pk: { A, B }, sk: { S } };
9 }
```

Listing 3: Mini-Frodo Key Generation

Mini-NTRU uses cyclic polynomial multiplication:

```
1 function polyMul(a: number[], b: number[]): number[] {
2   const result = new Array(N).fill(0);
3   for (let i = 0; i < N; i++) {
4     for (let j = 0; j < N; j++) {
5       const k = (i + j) % N; // Cyclic: x^N = 1
6       result[k] = (result[k] + a[i] * b[j]) % Q;
7     }
8   }
9   return result;
10 }
```

Listing 4: Mini-NTRU Polynomial Multiplication

4.4 E2EE Protocol Design

The end-to-end encryption protocol is the core mechanism that enables secure use of the PQC algorithms. It follows a three-message handshake that establishes a shared secret using the

user's selected algorithm (Kyber, Frodo, NTRU, or ECDH), then derives symmetric keys for message encryption.

Protocol Flow:

1. **E2EE_HELLO** (Initiator → Server → Responder):

- Initiator selects algorithm (e.g., “kyber”)
- Initiator calls `keyGen()` to generate a fresh key pair (pk, sk)
- Secret key sk is stored in browser `localStorage` (never leaves the device)
- Public key pk and algorithm identifier are sent to responder via server

2. **E2EE_KEY** (Responder → Server → Initiator):

- Responder receives initiator's public key and algorithm
- Responder calls `encaps(pk)` to generate ciphertext ct and shared secret K
- Responder generates random salt for key derivation
- Ciphertext and salt are sent back; shared secret K stays local
- Responder derives AES key from K using HKDF and stores session

3. **E2EE_MSG** (Bidirectional encrypted messages):

- Initiator receives ciphertext, calls `decaps(sk, ct)` to recover K
- Initiator derives same AES key using HKDF with matching salt/info
- Both parties now share identical AES-256-GCM key
- All subsequent messages encrypted with AES-GCM (random IV per message)

Key Derivation: HKDF (RFC 5869) derives the AES key from the shared secret with conversation-specific context to ensure domain separation—even if two conversations used the same PQC keys, their AES keys would differ:

```
1 const info = new TextEncoder().encode(  
2   'messaging-e2ee-v1:${algorithm}:conv:${conversationId}'  
3 );  
4 const aesKey = await crypto.subtle.deriveKey(  
5   { name: "HKDF", salt, info, hash: "SHA-256" },  
6   sharedSecretKey,  
7   { name: "AES-GCM", length: 256 },  
8   false,  
9   ["encrypt", "decrypt"]  
10 );
```

Listing 5: HKDF Key Derivation

Security Properties: The server only ever sees public keys and ciphertexts—it cannot decrypt messages or recover session keys. This is true E2EE: compromise of the server reveals nothing about message contents. The choice of PQC algorithm affects only the key exchange phase; once established, all algorithms provide identical AES-256-GCM message security.

4.5 Benchmarking Infrastructure

The benchmarking system provides two complementary modes:

Real-time Event Recording: Every cryptographic operation during normal application use is timestamped and logged, enabling analysis of actual usage patterns.

Synthetic Benchmarking: Configurable iteration counts (100–5,000) with warmup phases to account for JavaScript JIT compilation. Results include mean, standard deviation, minimum, maximum, and percentiles.

Results can be exported to CSV for external statistical analysis and visualisation.

5 Evaluation and Results

5.1 Benchmarking Methodology

All benchmarks were conducted on a MacBook Pro with Apple M2 chip and 16GB RAM, using Safari. Two distinct benchmark suites were executed:

KEM Operation Benchmark: Isolated measurement of individual cryptographic operations across 5,000 iterations per algorithm. This high iteration count ensures statistical significance and allows JIT compilation to stabilise, providing accurate average timings.

Session Throughput Benchmark: End-to-end simulation of realistic messaging sessions—500 sessions, each exchanging 100 messages of 500KB each. This extreme configuration required approximately 30 minutes of continuous execution but provides highly accurate readings for real-world performance assessment.

The methodology for both benchmarks:

- **Warmup Phase:** 50 iterations discarded to allow JIT optimisation
- **Timing:** `performance.now()` with microsecond resolution
- **Statistical Analysis:** Mean, standard deviation, min/max computed per operation
- **Isolation:** Tests run in dedicated browser tab with no other activity

5.2 KEM Operation Performance

Table 2 presents the results from 5,000 iterations of each KEM operation.

Table 2: KEM Operation Performance (5,000 iterations)

Operation	Mini-Kyber	Mini-Frodo	Mini-NTRU
KeyGen (avg)	$3.40 \mu\text{s} \pm 18.12 \mu\text{s}$	$11.60 \mu\text{s} \pm 32.02 \mu\text{s}$	$7.00 \mu\text{s} \pm 25.51 \mu\text{s}$
Encapsulate (avg)	$134.20 \mu\text{s} \pm 69.07 \mu\text{s}$	$129.80 \mu\text{s} \pm 124.79 \mu\text{s}$	$181.00 \mu\text{s} \pm 81.85 \mu\text{s}$
Decapsulate (avg)	$36.60 \mu\text{s} \pm 48.99 \mu\text{s}$	$38.20 \mu\text{s} \pm 49.00 \mu\text{s}$	$99.00 \mu\text{s} \pm 119.91 \mu\text{s}$
Total KEM (avg)	$174.20 \mu\text{s}$	$179.60 \mu\text{s}$	$287.00 \mu\text{s}$

Table 3 presents the key and ciphertext sizes for each algorithm.

Table 3: Key and Ciphertext Sizes

Parameter	Mini-Kyber	Mini-Frodo	Mini-NTRU
Public Key	86 bytes	104 bytes	27 bytes
Secret Key	26 bytes	25 bytes	25 bytes
Ciphertext	87 bytes	41 bytes	84 bytes
Shared Secret	32 bytes	32 bytes	32 bytes

The results demonstrate clear performance differentiation across algorithms. **Mini-Kyber achieves the fastest total KEM time** (174.20 μ s), followed by **Mini-Frodo** (179.60 μ s), with **Mini-NTRU** being slowest (287.00 μ s). For key generation specifically, Kyber is fastest (3.40 μ s), followed by NTRU (7.00 μ s), then Frodo (11.60 μ s). This ranking—Kyber, NTRU, Frodo for key generation—reflects the computational complexity of each algorithm’s key generation procedure, with Kyber’s Ring-LWE structure enabling particularly efficient polynomial operations.

5.3 Session Throughput Analysis

To obtain highly accurate real-world performance metrics, an extensive throughput benchmark was executed: 500 sessions, each exchanging 100 messages of 500KB. This extreme test ran for approximately 30 minutes of continuous operation, ensuring that timing variations are averaged out and the results reflect true steady-state performance.

Table 4: Session Throughput (500 sessions $\times$ 100 msgs $\times$ 500KB)

Metric	Mini-Kyber	Mini-Frodo	Mini-NTRU	ECDH
Key Exchange	836.00 μ s	782.00 μ s	2.22 ms	1.36 ms
Encrypt (per msg)	5.39 ms	5.55 ms	5.54 ms	5.53 ms
Decrypt (per msg)	1.90 ms	1.84 ms	1.84 ms	1.83 ms
Total Session	730.81 ms	741.14 ms	740.52 ms	739.84 ms

Table 5: PQC Overhead vs Traditional ECDH

Comparison	Overhead
Kyber vs ECDH	−1.2% (faster)
Frodo vs ECDH	+0.2%
NTRU vs ECDH	+0.1%

5.4 Analysis and Interpretation

The benchmarking results reveal several important insights that directly inform practical PQC deployment decisions:

1. PQC Overhead is Negligible in Practice: The most striking finding is that all three PQC algorithms perform within 1.2% of traditional ECDH in total session time. Mini-Kyber actually *outperforms* ECDH by 1.2%, demonstrating that post-quantum security does not require sacrificing performance. This result strongly supports the feasibility of PQC migration in real-world messaging applications.

2. Key Exchange Dominates PQC Overhead: The overhead difference between PQC and ECDH is concentrated entirely in the key exchange phase. NTRU’s key exchange (2.22 ms) is notably slower than Kyber (836 μ s) and Frodo (782 μ s). However, once session keys are established, all algorithms use identical AES-256-GCM encryption, resulting in nearly identical per-message encryption times ($\approx$ 5.5 ms) and decryption times ($\approx$ 1.85 ms) across all algorithms. This is the critical insight: *the choice of PQC algorithm only affects session establishment, not ongoing message throughput.*

3. Consistent Performance Ranking: Across both isolated KEM benchmarks and full session throughput, the algorithms consistently rank as:

1. **Mini-Kyber** — fastest overall (174.20 μ s KEM, 730.81 ms session), also fastest key generation (3.40 μ s)
2. **Mini-NTRU** — middle performance (287.00 μ s KEM, 740.52 ms session), second-fastest key generation (7.00 μ s)
3. **Mini-Frodo** — slowest KEM (179.60 μ s), but competitive session time (741.14 ms), slowest key generation (11.60 μ s)

Kyber’s consistent leadership validates NIST’s selection of Kyber (now ML-KEM) as the primary post-quantum KEM standard. The Ring-LWE algebraic structure that underpins Kyber enables efficient polynomial operations that outperform both Frodo’s matrix-based LWE and NTRU’s cyclic convolution approach.

4. Size-Performance Trade-offs: Interestingly, NTRU offers the smallest public key (27 bytes vs 86–104 bytes for Kyber/Frodo) and smallest ciphertext (84 bytes), which may be advantageous in bandwidth-constrained environments such as IoT or satellite communications, despite its slower computational performance.

5. Statistical Variance and Real-World Reliability: The high standard deviations observed (particularly Frodo’s encapsulation at $\pm$ 124.79 μ s and NTRU’s decapsulation at $\pm$ 119.91 μ s) reflect JavaScript’s JIT compilation and garbage collection behaviour. Notably, Frodo exhibits the highest maximum encapsulation time (2.70 ms outlier), while NTRU shows occasional decapsulation spikes (up to 2.50 ms). The 5,000-iteration KEM benchmark and 30-minute throughput test were specifically designed to average out these variations, ensuring the reported means accurately reflect steady-state performance rather than outlier behaviour.

5.5 Limitations

Several factors limit the generalisability of these results:

- **Simplified Parameters:** Mini implementations use reduced parameters for educational feasibility; production implementations (e.g., ML-KEM-768) would show different absolute timings but similar relative performance patterns
- **Browser Environment:** JavaScript execution differs from native C/assembly implementations; production libraries like liboqs would likely show faster absolute times
- **No Hardware Acceleration:** Production implementations leverage CPU vector instructions (AVX2, NEON) unavailable in browser JavaScript; Apple M2’s NEON capabilities remain unutilised
- **Single Platform:** Results are specific to Safari on Apple M2; different browsers and architectures may show varying characteristics

6 Critical Evaluation

6.1 Achievement of Objectives

The project successfully delivered:

- Fully functional real-time messaging application with user authentication
- Three PQC algorithm implementations (Mini-Kyber, Mini-Frodo, Mini-NTRU) plus ECDH baseline
- True end-to-end encryption with client-side cryptography
- Comprehensive benchmarking infrastructure with statistical analysis
- OpenAI GPT integration demonstrating PQC in AI communication contexts
- Educational implementations preserving algorithmic structure for learning

6.2 Reflection on Development Process

The project evolved significantly from its initial conception. Key lessons learned:

Framework Selection: The JHipster pivot, while initially frustrating, ultimately led to a cleaner, more maintainable architecture. The lesson: comprehensive frameworks are valuable for standard applications but can impede projects with non-standard requirements.

Scope Management: The decision to implement Mini versions rather than abandoning PQC entirely preserved the project’s core educational and research value. Recognising when to simplify while maintaining essential properties is a crucial engineering skill.

Architecture Decisions: The backend-to-frontend cryptography migration, while time-intensive, fundamentally improved the system’s security model. This experience reinforced the importance of security architecture decisions early in the design process.

6.3 Future Improvements

- **Hybrid Mode:** Implement combined classical/PQC key exchange for defence-in-depth
- **WebAssembly Integration:** Compile production libraries (liboqs) to WebAssembly for representative performance
- **Formal Verification:** Apply formal methods to verify implementation correctness
- **Group Messaging:** Extend E2EE protocol to multi-party conversations

7 Conclusion

This project demonstrates the practical feasibility of integrating post-quantum cryptography into web-based messaging applications. Despite evolving significantly from its initial JHipster/full-Kyber conception, the final system—featuring three Mini PQC implementations, true end-to-end encryption, comprehensive benchmarking, and AI integration—exceeds the original scope while maintaining technical rigour.

The empirical results strongly support an optimistic view of PQC adoption. The extensive benchmarking—5,000 KEM iterations and 30 minutes of continuous throughput testing across

500 sessions—reveals that post-quantum algorithms can actually *outperform* traditional ECDH in real-world scenarios. Mini-Kyber achieved the fastest total session time (730.81 ms), beating ECDH (739.84 ms) by 1.2%. All three PQC algorithms performed within 1.2% of the classical baseline, demonstrating that quantum resistance does not require performance sacrifice.

The consistent performance ranking—Kyber fastest, followed by NTRU, then Frodo—provides actionable guidance for developers choosing PQC algorithms. Kyber’s leadership in both isolated KEM operations (174.20 μ s) and full session throughput (730.81 ms) validates NIST’s selection of Kyber as the primary post-quantum standard. The critical insight is that PQC overhead concentrates entirely in key exchange; once session keys are established, message encryption is identical across all algorithms. For typical messaging patterns with long-lived sessions, the transition to quantum-resistant cryptography is practically achievable without any perceptible impact on user experience.

As quantum computing advances from theoretical threat to practical reality, the algorithms and implementation patterns explored in this project will become increasingly critical. This work contributes both a practical demonstration and educational resource for the cryptographic transition ahead, providing evidence that the post-quantum future need not compromise the performance users expect from modern secure messaging.

References

- [1] P. W. Shor, “Algorithms for Quantum Computation: Discrete Logarithms and Factoring,” *Proceedings 35th Annual Symposium on Foundations of Computer Science*, pp. 124–134, IEEE, 1994.
- [2] National Institute of Standards and Technology, “Module-Lattice-Based Key-Encapsulation Mechanism Standard (FIPS 203),” U.S. Department of Commerce, 2024.
- [3] O. Regev, “On Lattices, Learning with Errors, Random Linear Codes, and Cryptography,” *Journal of the ACM*, vol. 56, no. 6, pp. 1–40, 2009.
- [4] V. Lyubashevsky, C. Peikert, and O. Regev, “On Ideal Lattices and Learning with Errors over Rings,” *Advances in Cryptology – EUROCRYPT 2010*, pp. 1–23, Springer, 2010.
- [5] J. Hoffstein, J. Pipher, and J. H. Silverman, “NTRU: A Ring-Based Public Key Cryptosystem,” *Algorithmic Number Theory (ANTS III)*, pp. 267–288, Springer, 1998.
- [6] Signal Foundation, “The PQXDH Key Agreement Protocol,” Signal Technical Documentation, 2023.
- [7] J. W. Bos et al., “CRYSTALS-Kyber: A CCA-Secure Module-Lattice-Based KEM,” *2018 IEEE European Symposium on Security and Privacy*, pp. 353–367, 2018.